

WOXSEN UNIVERSITY

School of Technology

Design and Implementation of a Database Management System for Restaurant Table Reservation and Food Service Management System



DATABASE MANAGEMENT SYSTEMS

Course Project Report

B.Tech — Artificial Intelligence and Machine Learning

Second Year | Academic Year 2026–2027

Group member	Roll number
Y. Nikhil	25WU0102306
Y. Rupesh	25WU0102305
V. Sailesh	25WU0102301

Faculty: Dr.Kiranmayee Advala

Section: AIML Wolves

Date: 7 October 2026

2. Abstract

Restaurants handle several connected activities at the same time: customer registration, table availability, reservations, menu management and food orders. When these activities are handled through registers, calls or separate records, it becomes difficult to maintain a single and reliable view of availability and service status.

The proposed project develops a database-driven restaurant management system that brings these records together in a centralized relational database.

The system is designed around Customer, Restaurant_Table, Reservation, Menu, Food_Order and Order_Item entities. A customer can make reservations for a selected date and time, while restaurant staff can manage tables, reservations, menu items and food-service records. The design emphasizes primary keys, foreign keys, integrity constraints, useful joins and normalized relations.

The report follows the course-project structure specified by Woxsen University: abstract; introduction and problem statement; objectives and scope; requirements; ER diagram; relational schema and normalization; data dictionary; SQL commands; queries; UI design; implementation details; testing; conclusion and future enhancements; references; member contribution; and appendix.

Keywords: restaurant management, table reservation, MySQL, relational database, food ordering, DBMS.

3. Introduction and Problem Statement

A restaurant reservation process involves more than simply assigning a table. Customer details, table capacity, reservation time, menu information and food orders are connected and need to remain consistent. The supplied project presentation identifies manual booking, uncertain availability, duplicate reservations and difficulty in maintaining customer and reservation records as key problems.

The proposed solution is a centralized database-backed application. The basic workflow is: Customer → Website → Select Table → Date & Time → Confirm Reservation → Database. The customer side can support registration, viewing available tables, selecting a date and time, choosing a table and viewing reservation details. The administrative side can manage tables, reservations, customer information, menu items, food-service records and table availability.

Problem Statement

- Manual table booking is time-consuming and makes availability difficult to track.
- Customers may not have a clear view of available tables at a selected time.
- Multiple customers may attempt to reserve the same table for the same slot.
- Customer and reservation records are harder to maintain when stored manually.
- Restaurant staff need a common place to manage reservations and food-service information.
- Separate records can introduce duplication, inconsistent updates and avoidable errors.

Therefore, the project aims to design and implement a database management system that provides an efficient, reliable and organized solution for restaurant table reservation and food-service management.

4. Objectives and Scope

Objective	How the system addresses it
Online reservation	Provides a structured process for selecting a date, time and table.
Customer records	Stores customer details once and links them to reservations.
Table availability	Maintains table capacity and status so staff can identify available tables.
Conflict reduction	Uses reservation keys/constraints and application checks to reduce double booking.
Menu management	Stores food items, categories, prices and availability.
Food-service management	Links orders to reservations and menu items.
Reduced manual work	Centralizes records and makes retrieval through SQL queries easier.

Scope

The project focuses on a restaurant with customers, tables, reservations and food-service records. The database supports the core information needed by customers and restaurant administrators. Online payment, SMS/email notifications, customer reviews and advanced analytics are treated as future enhancements.

Intended users

- Customers: register, view availability, choose a table, make reservations and view reservation details.
- Restaurant staff/admin: manage tables, reservations, customer information, menu items and food-service records.
- Management: retrieve summarized information for operational decisions.

5. Software and Hardware Requirements

Layer	Requirement / role
Frontend	HTML, CSS and JavaScript for the web interface.
Backend	PHP, Python or Java, depending on the team's final implementation.
Database	MySQL relational database.
Server	XAMPP / Apache for a local deployment where applicable.
DB tool	MySQL Workbench or an equivalent MySQL client.
Hardware	Laptop/desktop capable of running a browser, application server and MySQL.
Connectivity	Localhost or network access between the UI/application and database server.

System Architecture

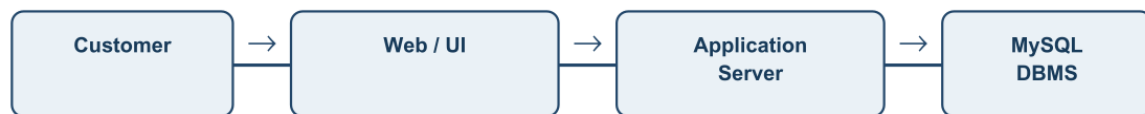


Figure 1. High-level architecture for the proposed restaurant reservation and food-service system.

The presentation describes the architecture as Customer → Table Reservation Website → Application Server → DBMS.

The database stores the customer, reservation, table, menu and food-order information so that related records can be retrieved and updated consistently.

6. ER Diagram

The ER model expands the entities named in the supplied presentation into a normalized structure. Order_Item is included as an additional associative entity so that one food order can contain multiple menu items without repeating columns or storing a list of items in a single field.

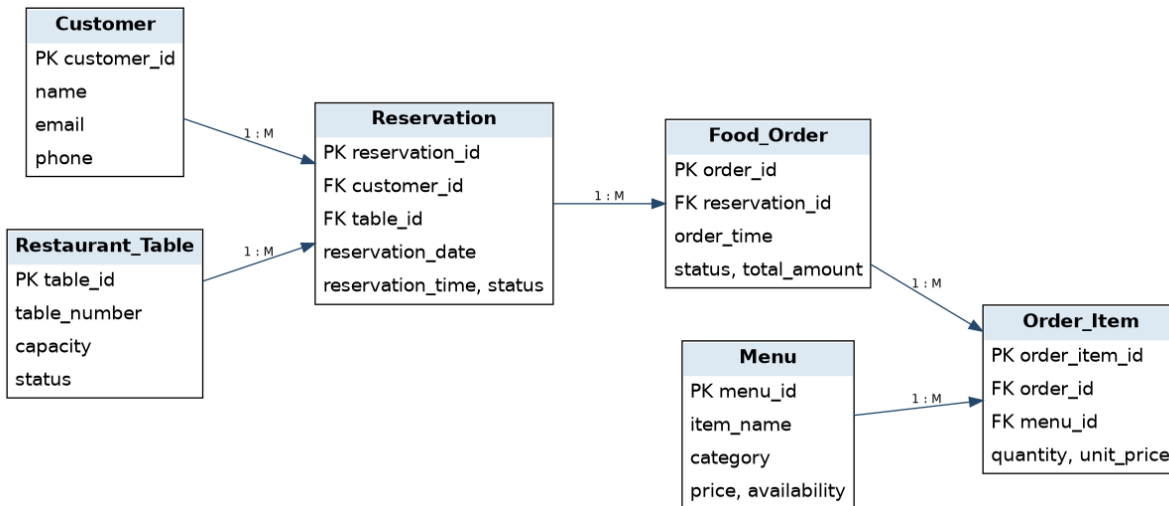


Figure 2. ER diagram for the restaurant table reservation and food-service database.

Relationship	Cardinality / participation
Customer – Reservation	One customer can make many reservations; every reservation belongs to one customer.
Restaurant_Table – Reservation	One table can appear in many reservations over time; every reservation selects one table.
Reservation – Food_Order	One reservation may have zero or many food orders; each order belongs to one reservation.
Food_Order – Order_Item	One order contains one or many order-item rows; each item row belongs to one order.
Menu – Order_Item	One menu item may appear in many order-item rows; each order-item row refers to one menu item.

7. Relational Schema and Normalization

Relation	Relational schema
Customer	Customer(customer_id PK, name, email UNIQUE, phone)
Restaurant_Table	Restaurant_Table(table_id PK, table_number UNIQUE, capacity, status)
Reservation	Reservation(reservation_id PK, customer_id FK, table_id FK, reservation_date, reservation_time, guest_count, status)
Menu	Menu(menu_id PK, item_name, category, price, availability)
Food_Order	Food_Order(order_id PK, reservation_id FK, order_time, status, total_amount)
Order_Item	Order_Item(order_item_id PK, order_id FK, menu_id FK, quantity, unit_price)

7.1 First Normal Form (1NF)

Each relation contains atomic values and avoids repeating groups. Menu items are not stored as a comma-separated list inside Food_Order; instead, each selected item is represented by a separate Order_Item row.

7.2 Second Normal Form (2NF)

The design uses single-attribute primary keys for the main entities. Order_Item also uses its own identifier, so every non-key attribute depends on the whole row identifier. The design therefore avoids partial dependency.

7.3 Third Normal Form (3NF)

Non-key attributes are kept with the entity they describe. Customer details remain in Customer, table details remain in Restaurant_Table and menu details remain in Menu. Reservation references the customer and table by foreign keys rather than repeating their descriptive attributes.

Design note: total_amount can be calculated from Order_Item rows. If it is stored for faster display, the implementation should keep it synchronized with the order items.

8. Data Dictionary

Customer

Field	Type	Constraints
customer_id	INT	PK, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
phone	VARCHAR(20)	NOT NULL

Restaurant_Table

Field	Type	Constraints
table_id	INT	PK, AUTO_INCREMENT
table_number	VARCHAR(10)	NOT NULL, UNIQUE
capacity	INT	NOT NULL
status	VARCHAR(20)	Available / Reserved / Maintenance

Reservation

Field	Type	Constraints
reservation_id	INT	PK, AUTO_INCREMENT
customer_id	INT	FK → Customer
table_id	INT	FK → Restaurant_Table
reservation_date	DATE	NOT NULL
reservation_time	TIME	NOT NULL
guest_count	INT	NOT NULL
status	VARCHAR(20)	Pending / Confirmed / Cancelled

Menu

Field	Type	Constraints
menu_id	INT	PK, AUTO_INCREMENT
item_name	VARCHAR(100)	NOT NULL
category	VARCHAR(50)	NOT NULL
price	DECIMAL(10,2)	NOT NULL
availability	BOOLEAN	DEFAULT TRUE

Food_Order

Field	Type	Constraints
order_id	INT	PK, AUTO_INCREMENT
reservation_id	INT	FK → Reservation

order_time	DATETIME	NOT NULL
status	VARCHAR(20)	Pending / Preparing / Served / Cancelled
total_amount	DECIMAL(10,2)	Derived / maintained by application

Order_Item

Field	Type	Constraints
order_item_id	INT	PK, AUTO_INCREMENT
order_id	INT	FK → Food_Order
menu_id	INT	FK → Menu
quantity	INT	NOT NULL
unit_price	DECIMAL(10,2)	NOT NULL

9. SQL Commands Used (DDL, DML) with Sample Outputs

9.1 Database and table creation

```
CREATE DATABASE IF NOT EXISTS restaurant_management;  
USE restaurant_management;
```

```
CREATE TABLE Customer (  
    customer_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    phone VARCHAR(20) NOT NULL  
);
```

```
CREATE TABLE Restaurant_Table (  
    table_id INT AUTO_INCREMENT PRIMARY KEY,  
    table_number VARCHAR(10) NOT NULL UNIQUE,  
    capacity INT NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'Available'  
);
```

9.2 Reservation table and foreign keys

```
CREATE TABLE Reservation (  
    reservation_id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_id INT NOT NULL,  
    table_id INT NOT NULL,  
    reservation_date DATE NOT NULL,  
    reservation_time TIME NOT NULL,  
    guest_count INT NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'Confirmed',  
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),  
    FOREIGN KEY (table_id) REFERENCES Restaurant_Table(table_id)  
);
```

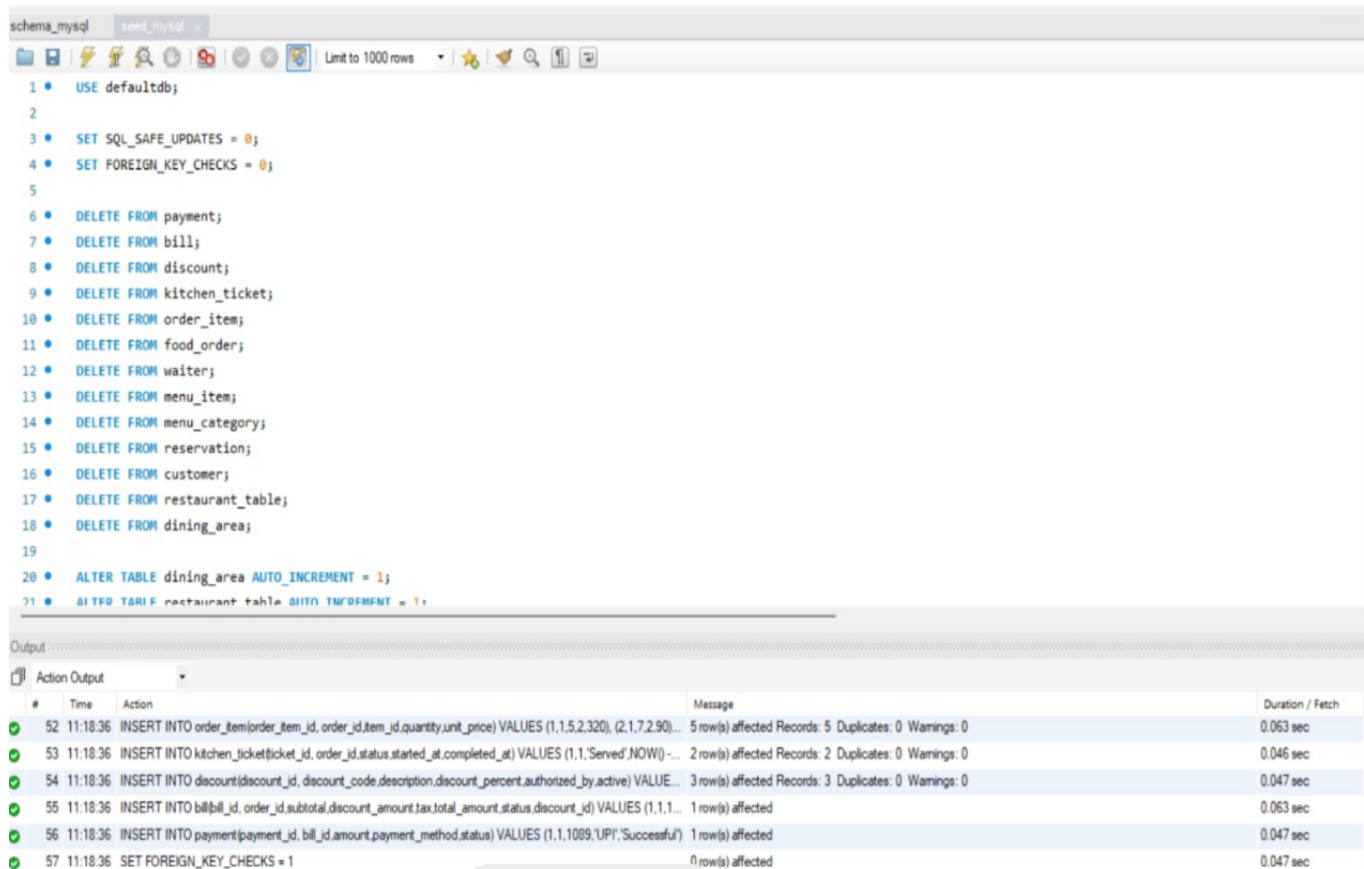
9.3 Sample DML

```
INSERT INTO Customer (name,email,phone)  
VALUES ('Aarav Kumar','aarav@example.com','9876543210');
```

```
INSERT INTO Restaurant_Table (table_number,capacity,status)  
VALUES ('T01',4,'Available');
```

```
INSERT INTO Reservation  
(customer_id,table_id,reservation_date,reservation_time,guest_count,status)  
VALUES (1,1,'2026-10-10','19:30:00',4,'Confirmed');
```

The commands above are representative SQL prepared from the proposed schema. They should be synchronized with the group's final .sql file before submission.



```
1 • USE defaultdb;
2
3 • SET SQL_SAFE_UPDATES = 0;
4 • SET FOREIGN_KEY_CHECKS = 0;
5
6 • DELETE FROM payment;
7 • DELETE FROM bill;
8 • DELETE FROM discount;
9 • DELETE FROM kitchen_ticket;
10 • DELETE FROM order_item;
11 • DELETE FROM food_order;
12 • DELETE FROM waiter;
13 • DELETE FROM menu_item;
14 • DELETE FROM menu_category;
15 • DELETE FROM reservation;
16 • DELETE FROM customer;
17 • DELETE FROM restaurant_table;
18 • DELETE FROM dining_area;
19
20 • ALTER TABLE dining_area AUTO_INCREMENT = 1;
21 • ALTER TABLE restaurant_table AUTO_INCREMENT = 1;
```

Output

#	Time	Action	Message	Duration / Fetch
52	11:18:36	INSERT INTO order_item(order_item_id, order_id,item_id,quantity,unit_price) VALUES (1,1,5,2,320), (2,1,7,2,90)...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.063 sec
53	11:18:36	INSERT INTO kitchen_ticket(ticket_id, order_id,status,started_at,completed_at) VALUES (1,1,'Served',NOW())...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.046 sec
54	11:18:36	INSERT INTO discount(discount_id, discount_code,description,discount_percent,authorized_by,active) VALUE...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.047 sec
55	11:18:36	INSERT INTO bill(bill_id, order_id,subtotal,discount_amount,tax,total_amount,status,discount_id) VALUES (1,1,1...	1 row(s) affected	0.063 sec
56	11:18:36	INSERT INTO payment(payment_id, bill_id,amount,payment_method,status) VALUES (1,1,1089,'UPI','Successful')	1 row(s) affected	0.047 sec
57	11:18:36	SET FOREIGN_KEY_CHECKS = 1	0 row(s) affected	0.047 sec

Figure 1sql operations

```

Restaurant_DBMS_MySQL_Complete > app.py > ...
383 def reservations():
471     (
472         request.form["customer_id"],
473         table_id,
474         request.form["reservation_date"],
475         request.form["reservation_time"],
476         guest_count,
477         request.form["status"]
478     )
479 )
480
481 if request.form["status"] == "Confirmed":
482     cur.execute(
483         """
484         UPDATE restaurant_table
485
486         SET status='Reserved'
487
488         WHERE table_id=%s
489         """
490         ,
491         (table_id,)
492     )
493
494     conn.commit()
495
496     flash(
497         "Reservation created successfully.",
498         "success"
499     )
500
501 except Error as e:
502     conn.rollback()
503
504     flash(
505

```

Figure 2 vs code

10. Queries with Outputs

10.1 View available tables

```
SELECT table_number, capacity
FROM Restaurant_Table
WHERE status = 'Available';
```

Illustrative sample output:

table_number	capacity
T01	4
T03	2
T05	6

10.2 Customer reservation details

```
SELECT c.name, r.reservation_date, r.reservation_time,
       t.table_number, r.status
FROM Customer c
JOIN Reservation r ON c.customer_id = r.customer_id
JOIN Restaurant_Table t ON r.table_id = t.table_id;
```

Illustrative sample output:

name	reservation_date	reservation_time	table_number	status
Aarav Kumar	2026-10-10	19:30	T01	Confirmed
Meera Shah	2026-10-10	20:00	T03	Confirmed

10.3 Orders with menu items

```
SELECT f.order_id, m.item_name, oi.quantity, oi.unit_price
FROM Food_Order f
JOIN Order_Item oi ON f.order_id = oi.order_id
JOIN Menu m ON oi.menu_id = m.menu_id;
```

Illustrative sample output:

order_id	item_name	quantity	unit_price
101	Paneer Tikka	2	240.00
101	Butter Naan	4	55.00

10.4 Aggregate query – reservations per table

```
SELECT table_id, COUNT(*) AS reservation_count  
FROM Reservation  
GROUP BY table_id;
```

Illustrative sample output:

table_id	reservation_count
1	3
2	2
3	4

11. UI Design and Screens

The supplied presentation proposes a web-based interface with separate customer and admin responsibilities. The following layout is a clean reference design for the report. Actual application screenshots should be inserted in the final submission if the working UI is available.

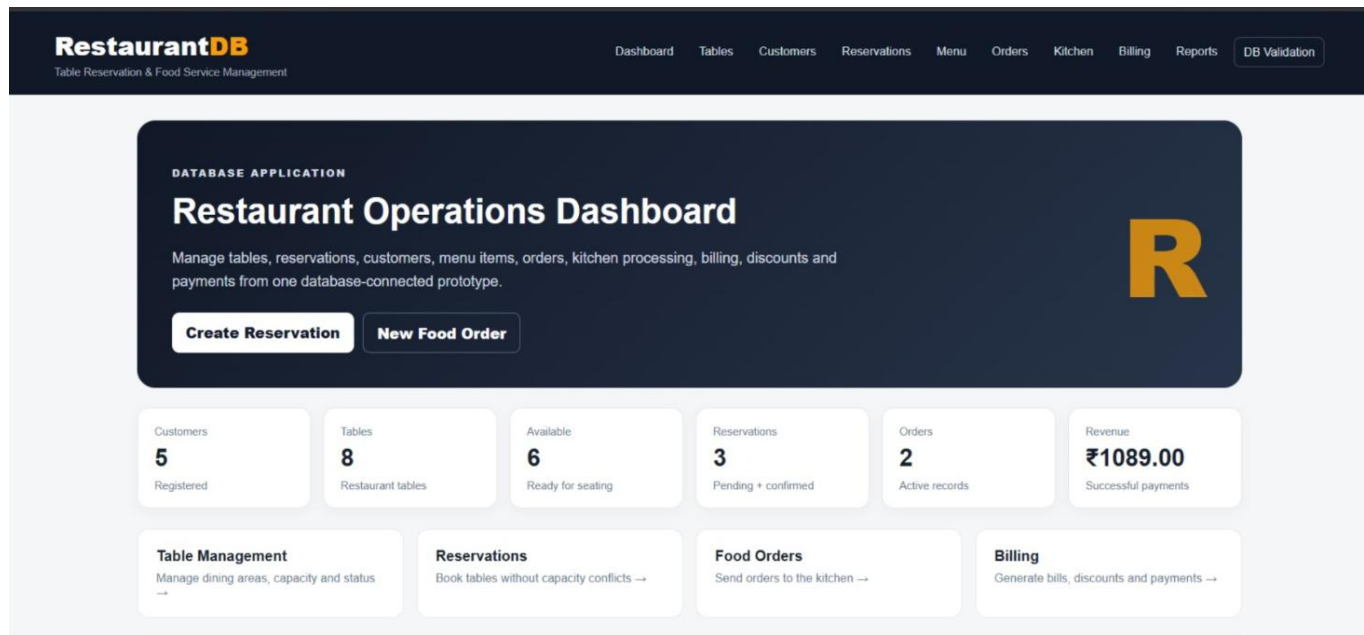


Figure 3. Illustrative customer reservation interface.

Screen	Main functions
Customer registration/login	Create or access a customer account.
Table availability	View table number, capacity and availability.
Reservation form	Select date, time, number of guests and table.
Reservation details	View confirmed reservation information.
Admin dashboard	Manage customers, tables, reservations, menu and food orders.
Order management	Add menu items to an order and update order status.

12. Implementation Details

12.1 Technology Stack

Component	Planned / proposed use
HTML + CSS	Page structure and visual presentation.
JavaScript	Client-side validation and interactive table/reservation actions.
PHP / Python / Java	Server-side application logic, based on the team's final implementation.
MySQL	Central relational database for persistent records.
XAMPP / Apache	Local application-server environment where applicable.

12.2 Database Connectivity

The application layer should establish a connection to MySQL using the selected backend technology. User actions such as creating a reservation are translated into INSERT/UPDATE statements or parameterized database calls. Read operations use SELECT queries, while administrative actions update or delete records subject to foreign-key rules.

12.3 Key implementation logic

A reservation request should validate the customer, selected table, date, time and guest count before insertion. The application should check whether the chosen table is already reserved for the same date and time. The selected table's capacity should also be compared with the number of guests.

```
SELECT reservation_id
FROM Reservation
WHERE table_id = ?
AND reservation_date = ?
AND reservation_time = ?
AND status IN ('Pending','Confirmed');
```

13. Testing

Test case	Input / action	Expected result	Status
Customer insertion	Enter valid customer details	Customer row is inserted	To verify
Duplicate email	Use an existing email	UNIQUE constraint prevents duplicate	To verify
Table insertion	Add a valid table	Table appears in Restaurant_Table	To verify
Reservation	Select free table/date/time	Reservation is created	To verify
Double booking	Reserve same table/time twice	Second booking is rejected	To verify
Invalid customer FK	Use nonexistent customer_id	Foreign-key error / validation	To verify
Menu insertion	Add menu item with price	Item appears in Menu	To verify
Food order	Create order with items	Food_Order and Order_Item rows are stored	To verify
Join query	Run reservation/customer/table join	Combined reservation view is returned	To verify
Aggregate query	COUNT reservations by table	Grouped counts are displayed	To verify
Delete test	Delete a test record where permitted	Record is removed without orphaning data	To verify

The table intentionally marks execution as “To verify” because the supplied project presentation does not contain evidence of the final SQL execution or working UI. Before submission, replace these statuses with the actual results and attach screenshots showing before/after database and UI states.

14. Conclusion and Future Enhancements

The proposed restaurant table reservation and food-service management system demonstrates how a relational DBMS can organize connected restaurant activities in one place. The database separates customers, tables, reservations, menu items and food-order records while maintaining relationships through primary and foreign keys.

The design addresses the main issues identified in the supplied presentation: manual booking, uncertain availability, duplicate reservations and scattered records. A centralized database also makes joins, aggregation, reporting and controlled updates easier. The ER model and normalized relational schema provide a structured base for a web application.

Future enhancements

- Online payment integration with transaction history.
- SMS/email or WhatsApp reservation confirmations and reminders.
- Food pre-ordering during table reservation.
- Customer reviews and feedback management.
- Analytics for peak hours, popular menu items and table utilization.
- Role-based access control for manager, staff and customer accounts.
- Audit logs and stronger transaction handling for concurrent reservations.
- Mobile-responsive interface and deployment to a hosted database.

15. References

- [1] Woxsen University. Instructions for Final Presentation — DBMS Course Project: AIML Wolves, CSE Tigers, CSE Rhinos. October 2026, pp. 1–2.
- [2] Y. Nikhil, Y. Rupesh and V. Sailesh. Design and Implementation of a Database Management System for Restaurant Table Reservation and Food Service Management System. Supplied project presentation, October 2026.
- [3] Supplied DBMS Course Project Report. Design and Implementation of a Database Management System for Hostel Accommodation and Student Services. Used as the formatting and organizational reference.
- [4] Oracle. MySQL 8.0 Reference Manual. SQL statements, constraints, keys and relational database concepts.

Source-use note

The report content specific to the restaurant project is derived from the supplied project presentation. The section order and submission requirements follow the supplied Woxsen DBMS course instructions. The visual structure takes inspiration from the supplied hostel report without copying its project-specific content.

16. Contribution of Each Member

Member	Roll number	Suggested contribution
Y. Nikhil	25WU0102306	Database design, ER diagram, relational schema and SQL queries.
Y. Rupesh	25WU0102305	UI / application workflow, reservation module and testing.
V. Sailesh	25WU0102301	Menu and food-order module, documentation and presentation support.

The contribution split above is a structured project allocation prepared for the report. The group should edit it if the actual division of work differs. All three members should be prepared to explain every part of the project, as required by the course instructions.

17. Appendix: GitHub Repository Link

Create the repository according to the course instructions and place the project files inside the required folders. The repository should contain the presentation, project report, ER diagram, SQL file, UI source code and screenshots of the UI states.

Required item	Recommended repository location
Presentation PPT/PDF	/Presentation/
ER diagram image	/Presentation/ER-Diagram.png
SQL commands	/Presentation/restaurant_management.sql
UI source code	/Presentation/Source-Code/
UI screenshots	/Presentation/Screenshots/
Final report PDF	/Project-Report/
README.md	/README.md

GitHub Repository: _____

The actual repository URL should be entered here after the group creates and shares it.

End of Report